

Homework 3

Name: Diya Jain U81673988

1. What is the "same-origin policy"?

Answer: The "same-origin policy" is a crucial security concept enforced by web browsers. It's designed to prevent scripts running on one web page from accessing data or interacting with scripts on another page unless the two pages have the same origin.

An origin is defined by the combination of the protocol (like HTTP or HTTPS), the domain (the website address), and the port number if specified. If two web pages have the same protocol, domain, and port number, they are considered to have the same origin and can freely interact with each other.

This policy helps protect users from malicious attacks like Cross-Site Scripting (XSS) or Cross-Site Request Forgery (CSRF). Without the same-origin policy, a script from one site could potentially access sensitive data or perform actions on another site on behalf of the user, leading to security vulnerabilities and privacy breaches.

2. RJ, a web programmer for a bank, has been given the task of building the online banking web site. He needs to create an authentication web application that accepts user input of username and passwords and verifies the input. RJ is told to use authentication cookies so that the web application does not frequently re-authenticate users. He generates a random number S only once, and then increases S by a constant value of $0x5f$ to create the cookie for each user. For example, the authentication cookies for user 1, 2, 3, ... are $S + 0x5f$, $S + 0x5f + 0x5f$, $S + 0x5f + 0x5f + 0x5f$..., respectively. Please describe an attack that exploits RJ's implementation to compromise other users' accounts and discuss a possible solution to address this attack.

Answer: RJ's implementation of generating authentication cookies by incrementing a random number S by a constant value for each user poses a security vulnerability known as predictable cookie values.

An attacker could potentially exploit this by observing the pattern of cookie values. If an attacker gains access to their own account's authentication cookie, they could predict the cookies of other users by applying the same incremental pattern, especially if they can guess or obtain the initial random number S used for their own cookie.

For instance, if an attacker knows their own authentication cookie ($S + 0x5f$, for example) and recognizes the pattern ($S + 0x5f + 0x5f$, $S + 0x5f + 0x5f + 0x5f$, etc.), they can easily generate and test these values to access other users' accounts without needing their actual credentials. This is known as a predictable cookie attack.

A possible solution to address this vulnerability would be to make the authentication cookies more random and unpredictable. Instead of using a predictable increment, RJ could generate a unique, random value for each user's authentication cookie. Using a cryptographically secure random number generator to create a sufficiently random value for each user's cookie can prevent attackers from predicting or deducing other users' cookie values, significantly enhancing security. Additionally, implementing additional security measures like session expiration, secure cookie attributes, and monitoring for unusual or suspicious login activities can further bolster the overall security of the authentication system.

3. Textbook page 198: Consider a scenario in which an attacker is interested in breaking into Windows NT or UNIX shell accounts. Analyze the technical issues surrounding the password security that these operating systems offer. State your assumptions.

- a. How many bits of information are possible in an eight-character password if any character can be used? (Assume each character consists of 8 bits)

Answer: 64 bits²⁶⁺

- b. How many different combinations of passwords are there in an eight-character password if only the uppercase and lowercase characters plus the ten decimal digits can be used?

Answer: If only uppercase letters, lowercase letters, and ten decimal digits can be used, the character set size would be 26 (uppercase) + 26 (lowercase) + 10 (digits) = 62 characters. For an eight-character password using this limited character set, the total number of combinations would be 62^8 .

4. Would it be possible to thwart all possible password cracking attacks by increasing the salt length to something large, like 64 bits?

Answer: Increasing the salt length, such as to 64 bits, is a substantial step towards bolstering password security. Salting involves adding a unique random value to each password before hashing it, making precomputed tables like rainbow tables less effective against cracking attempts. A longer salt significantly expands the search space for attackers attempting to crack passwords. A 64-bit salt offers an immense number of possible combinations, which increases the computational complexity for attackers. This complexity makes brute-force attacks more challenging and time-consuming, as the number of possible combinations grows exponentially with the increase in salt length. However, relying solely on a longer salt length isn't a foolproof solution against all password cracking methods. Determined attackers equipped with powerful hardware or specialized systems can still attempt to crack passwords, especially if the passwords themselves are weak or easily guessable. Additionally, longer salts might not defend against dictionary-based attacks if passwords are common or easily found in dictionaries.

5. What is an Advanced Persistent Threat (APT)? Which tool we have discussed so far can be used to mitigate this threat? Please explain this tool.

Answer: Advanced Persistent Threats (APTs) are highly sophisticated and prolonged cyberattacks aiming to infiltrate networks for data theft or disruption. Intrusion Detection Systems (IDS), including Network-based (NIDS) and Host-based (HIDS), are crucial tools in combating these threats.

An IDS operates by continuously monitoring network traffic or system activities, detecting anomalies or known attack patterns in real-time. It alerts security teams upon identifying suspicious behavior, enabling immediate investigation and response.

Additionally, IDS offers anomaly detection, capable of identifying new or previously unseen threats beyond known attack signatures. Its 24/7 surveillance assists in identifying and thwarting ongoing threats posed by persistent APTs.

However, to effectively counter APTs, combining IDS with complementary security measures like firewalls, employee training, and comprehensive incident response plans is essential for a robust defense strategy.

6. Programmer RJ tries to design a network protocol to enable a client to check whether a server is still active. He creates a protocol in this way. The client generates a random string and sends the string together with the string length to the server. If the server is still active, the server will echo the string back to the client so that the client knows that the connection is not lost. The server uses the `string_process` function below to process the string. The function first creates a buffer of the size equal to `strlen`, which is the string length sent by the client, and then it copies the input string (the one sent by the client) of size `strlen` to the buffer. The `echo_string` function sends the content in the buffer to the client. This code has a vulnerability that can be exploited by an adversary to cause severe consequences. Please identify the vulnerability, discuss a solution, and explain your reason.

```

void string_process (unsigned char *string, unsigned int strlen)
{
    unsigned char *buffer;

    buffer = (unsigned char *) malloc (strlen);
    memcpy (buffer, string, strlen); echo_string
    (buffer, strlen); return;
}

```

Answer: The vulnerability in the 'string_process' function is a potential buffer overflow due to inadequate memory allocation. The 'malloc' function allocates memory for the buffer solely based on the 'strlen' parameter provided by the client, allowing the possibility of insufficient memory allocation if an excessively large 'strlen' value is sent.

To address this vulnerability, several steps should be taken:

1. Boundary Checking: Validate the 'strlen' value received from the client to ensure it doesn't exceed a predefined maximum limit. This prevents excessive allocation that could lead to buffer overflow.
2. Secure Memory Allocation: Allocate memory for the buffer using 'malloc' with a size slightly larger than 'strlen' to accommodate the received string and provide an additional safety margin, preventing potential buffer overflow issues.
3. Revised Function Implementation: Update the 'string_process' function to include these safety measures, ensuring proper memory allocation and validation of input length before processing the string.

By implementing these changes, the revised function can mitigate the risk of buffer overflow attacks, ensuring that the allocated buffer size is sufficient and preventing potential memory corruption or security vulnerabilities due to inadequate memory allocation.

7. If an organization uses network-based IDS to detect attacks, and it can monitor all incoming and outgoing traffic, is there any value to also using host-based IDS? If so, why, and if not, why not?

Answer: Employing both network-based IDS (NIDS) and host-based IDS (HIDS) in an organization's security strategy holds significant value due to their complementary roles and capabilities.

By combining NIDS and HIDS, organizations achieve a multi-layered security approach that enhances overall threat detection and response capabilities. NIDS provides a broader view of network traffic, capturing threats that traverse multiple hosts, while HIDS offers deeper visibility into the activities occurring within each individual host.

The synergy between these two types of IDS allows for better correlation and analysis of security events. Information from NIDS and HIDS can be correlated, enabling security teams to detect sophisticated attacks that involve multiple entry points or various attack vectors. This correlation helps in understanding the context of an attack, allowing for a more comprehensive and targeted response to potential security incidents.

Ultimately, the combined use of NIDS and HIDS ensures a more robust security posture by covering both network-wide threats and host-specific vulnerabilities. This approach strengthens an organization's ability to detect, respond to, and mitigate a wide range of cyber threats effectively, thereby fortifying the overall security infrastructure.

8. Eve, a hacker, uses some automatic tool to scan existing popular web pages to search for the SQL injection vulnerability. Eve finds a webpage with the URL below

`http://oceanbaynews.com/music.php?id=3153`

From the URL, Eve guesses that this URL sends a search query to some database to search a table of the name 'music'. The search query may look like this

`Select * from music where id = 3153`

Eve wants to know what information is stored in this database. So, he wants to know the names of all tables, and the names of all table columns. Please compose malicious URL with the SQL injection payload that can display this information to Eve. Please also propose a defense method to prevent this attack.

Answer: Here's an example of a malicious URL with a SQL injection payload that could potentially expose this information:

`http://oceanbaynews.com/music.php?id=3153' UNION SELECT table_name, column_name FROM information_schema.columns--`

Explanation of the Payload:

- The injected payload `' UNION SELECT table_name, column_name FROM information_schema.columns--` is concatenated to the original URL parameter (`'id=3153'`) after adding a single quote (`'`) to terminate the original SQL query.

- `'UNION SELECT'` is used to combine the original query with a new query that retrieves table names (`'table_name'`) and column names (`'column_name'`) from the `'information_schema.columns'` table, which typically stores metadata about the database structure.

- `--` is a comment to ensure that any remaining part of the original query is ignored to prevent syntax errors.

Defense Method to Prevent SQL Injection Attacks:

1. **Parameterized Queries or Prepared Statements:** Use parameterized queries or prepared statements in the application code to ensure that user inputs (such as the `'id'` parameter in the URL) are treated as data rather than executable code. This prevents attackers from injecting malicious SQL commands.
2. **Input Validation and Sanitization:** Implement strict input validation and data sanitization measures to validate and filter user inputs. Limit input to expected data types and lengths and reject any input that doesn't meet defined criteria.
3. **Escaping Special Characters:** Escape or sanitize special characters that can alter SQL syntax, such as single quotes (`'`), to neutralize their impact on SQL queries.

4. Least Privilege Principle: Ensure that database accounts used by the application have the least privileges required for their specific tasks. This limits the damage that an attacker can cause even if a successful SQL injection occurs.

By implementing these preventive measures, the application can significantly reduce the risk of SQL injection attacks, protecting the database from unauthorized access and manipulation of sensitive information.

9. Please construct a XSS attack to attack yourself, i.e., to steal your own cookie.

Welcome diyajain@usf.edu
Today is 2023-11-17 11:44:35

The cookie you can use to login the victim's account is PHPSESSID=137dkfeldjcenna2aqefgnuurk

congratulations! The first key you need to submit is
UEhQU0VTU0IEPTEzN2RrZmVsZGpjZW5uYTJhcWVmZ251dXJrMjAyMzExMTdhYjZiMzMxZQ%3D%3D

You can try to do the XSS attack against a victim user. The TA will play the role of a victim user.

Welcome diyajain@usf.edu
Today is 2023-11-19 21:49:02

The cookie you can use to login the victim's account is PHPSESSID=573c53ai8e0n2inh1di8j5m8ts

congratulations! The first key you need to submit is
UEhQU0VTU0IEPTU3M2M1M2FpOGUwbjJpbmgxZGk4ajVtOHRzMjAyMzExMjBhYjZiMzMxZQ%3D%3D

The second key you need to submit is: LTIwMjMtMTetMjAgMTE6MTI6MThwbS05ODY2NDg2NEJvdHRvbXNVcA%3D%3D

There are 22 files in this direcotry:

```
drwxrwxr-x 2 www www 4096 Mar 19 18:32 css/
drwxrwxr-x 2 www www 4096 Mar 19 12:50 keys/
-rw-rw-r-- 1 www www 686 Mar 26 16:47 getcookie.php
-rw-rw-r-- 1 www www 3248 Mar 25 09:13 home.php
drwxrwxr-x 2 www www 4096 Mar 19 12:41 imgs/
-rwxr--r-x 1 root root 193 Mar 17 23:36 catchFlag*
-rw-rw-r-- 1 www www 1646 Mar 25 09:13 login.php
-rw-rw-r-- 1 www www 3052 Mar 26 16:42 log.txt
-rwxrwxr-x 1 www www 15704 Mar 19 12:49 Run*
-rw-rwxr-x 1 www www 293 Mar 17 13:36 writekey*
-rwxr--r-x 1 admin www 193 Mar 14 23:36 target*
-rw-rw-r-- 1 www www 424 Mar 19 11:40 webshell.php
-rwx-rwxr-x 1 www www 16704 Mar 19 12:49 BottomsUp*
-rw-rw-r-- 1 www www 3248 Mar 25 09:19 index.php
-rwxrw-r-- 1 www www 3114 Mar 26 10:23 Hooked*
-rw-r--r-x 1 www www 188 Mar 14 21:36 generator*
-rw-rw-r-- 1 www www 3148 Mar 25 10:13 register.php
-rw-rw-r-- 1 www www 3228 Mar 25 19:13 fileread.php
-rw-rwxr-x 1 www www 314 Mar 25 10:23 WriteKey*
-rw-rw-r-- 1 www www 3238 Mar 25 09:33 filewrite.php
-rwx-rwxr-x 1 www www 16704 Mar 20 12:49 Sister*
-rw-rwxr-x 1 www www 293 Mar 17 23:36 readkey*
```